VERNEX SMART ATTENDANCE

Task Sheet: Super Admin Portal, Issue Reporting & Production Hardening

Prepared 13 October 2026 | Repo: github.com/tamizhv406/Vernex-Attendance-System | To be built with Claude Code in VS Code

#	Section	What it covers
0	How to use this sheet	Order of work, one phase per session, rules
1	Setup & run commands	Fresh laptop setup, every-day run, troubleshooting
2	Current state (audit findings)	What exists today and what blocks production
3	Phase 1 - Authentication & roles	Real login, accounts, protecting every API route
4	Phase 2 - Super Admin portal	Create / manage admins, dashboard, audit log
5	Phase 3 - Issue reporting	Admin reports a problem; it appears in the Super Admin portal
6	Phase 4 - Security hardening	OWASP-style checks, secrets, headers, uploads, biometrics, privacy
7	Phase 5 - Compatibility	Browsers, phones, screen sizes, runtimes
8	Phase 6 - Availability & reliability	Health, backups, restarts, deployment, monitoring
9	Phase 7 - Tests & CI	Automated tests and scans on every push
10	Master prompt & phase prompts	Copy-paste prompts for Claude Code
11	Manual checks	Hands-on checklist to sign off before go-live

No secrets in this document. Wherever a password or key is needed it is shown as <DB_PASSWORD>. Keep real values only in `backend\.env` (gitignored). The Supabase service-role key and DB password were shared in chat earlier - rotate both before go-live (task S-01).

0. How to use this sheet

- Work **phase by phase, in order** (1 -> 7). Phases 2 and 3 depend on Phase 1 (login and roles).
- Use **one Claude Code session per phase**. Start each with the Master Prompt (section 10), then the phase prompt.
- Ask Claude Code to **plan first** (plan mode / Shift+Tab), review the plan, then let it implement.
- After each phase: run the tests, do that phase's manual checks (section 11), then commit on a feature branch (e.g. `feature/phase-1-auth`) and open a pull request.
- Task IDs (A-xx, SA-xx, IR-xx, S-xx, C-xx, AV-xx, T-xx) are used in commits and in the manual-check list.

1. Setup & run commands

Prerequisites: **Python 3.10** (tick "Add python.exe to PATH"; TensorFlow does not install on 3.13), **Node.js 20+**, **Git**, about 3 GB free disk. Windows PowerShell commands.

1.1 One-time setup on a fresh laptop

```
git clone https://github.com/tamizhv406/Vernex-Attendance-System.git
cd Vernex-Attendance-System

# Backend
cd backend
py -3.10 -m venv .venv
.\.venv\Scripts\python -m pip install --upgrade pip
.\.venv\Scripts\python -m pip install -r requirements.txt
copy .env.example .env
```

Open `backend\.env` and add this line (no # in front). In the password write @ as `%40` and % as `%25`:

```
DATABASE_URL=postgresql://postgres:<DB_PASSWORD>@db.edxuzkdxqksbhsozqhlt.supabase.co:5432/postgres
```

If the connection fails on that network (the direct host needs IPv6), use the pooler instead:

```
DATABASE_URL=postgresql://postgres.edxuzkdxqksbhsozqhlt:<DB_PASSWORD>@aws-0-ap-southeast-2.pooler.supabase.co
```

```
# Test the database connection -> "Supabase PostgreSQL connection successful"
.\.venv\Scripts\python scripts\test_supabase_connection.py

# Frontend
cd ..\frontend
npm install
```

1.2 Every time you run the project (two terminals)

```
# Terminal 1 - backend (keep open)
cd Vernex-Attendance-System\backend
$env:PYTHONUTF8 = "1"
.\.venv\Scripts\python -m uvicorn app.main:app --host 0.0.0.0 --port 8000

# Terminal 2 - frontend (keep open)
cd Vernex-Attendance-System\frontend
npm run dev:https
```

- Wait until `http://localhost:8000/health` shows `"face_models":"ready"` (~30 s; first run downloads ~100 MB of models).
- PC: open **https://localhost:5173** (self-signed certificate -> Advanced -> Proceed).
- Phone (same Wi-Fi): the `https://<PC-WiFi-IP>:5173/attendance-scanner` link printed by the frontend.
- Shortcut after setup: double-click `start.bat`. If the backend shows *"face models failed to load ... 'charmap' codec"*, use the manual command above (it sets PYTHONUTF8).

1.3 Run the tests

```
cd backend;  .\.venv\Scripts\python -m pytest
cd ..\frontend;  npm test
```

1.4 Troubleshooting

Problem	Fix
pip fails on tensorflow / deepface	Wrong Python. Install 3.10, delete `backend\.venv`, redo 1.1.
'charmap' codec error, face models not ready	Set `$env:PYTHONUTF8="1"` before starting the backend.
DB connection fails / timeout	Use the pooler URL; check password encoding; Supabase project may be paused (free tier) - resume it in the dashboard.
Port 8000 / 5173 already in use	Another copy is running - close it or restart the PC.
Phone can't open page / camera	Same Wi-Fi, use https:// link, accept certificate, allow Node.js in Windows Firewall, allow camera permission.

2. Current state (audit findings, 13 Oct 2026)

Stack: FastAPI + SQLAlchemy + DeepFace (TensorFlow) backend on port 8000; React 19 + Vite 7 + Tailwind frontend on 5173, which proxies `/api` to the backend. Database: Supabase PostgreSQL 17 (Sydney, free tier). Tables are created/upgraded automatically on startup (`app/schema.py`); `backend/migrations/full_schema.sql` holds the full current schema.

Severity	Finding	Where
Critical	Login page does not check anything - any input opens the console. No accounts table, no passwords.	frontend/src/pages/Login.jsx
Critical	No API authentication: every endpoint (people, faces, attendance, uploads, deletes) is open to anyone who can reach port 8000 or the Vite proxy.	backend/app/routes/*
Critical	Supabase service-role key and DB password were shared in plain text (chat). Must be rotated.	Supabase dashboard
High	Global error handler returns `str(exc)` to the client - leaks SQL/internal details.	backend/app/main.py
High	CORS allows any private-network origin with credentials (regex for 192.168.*, 10.*, 172.16-31.*).	backend/app/main.py
High	Backend connects as the `postgres` superuser (bypasses RLS). Face embeddings are biometric data.	backend/.env
High	Production would run on the Vite dev server with a self-signed certificate; no real TLS, no process supervision.	start.bat, vite.config.js
Medium	`/docs` and `/openapi.json` are public; unpinned Python dependencies; deprecated `@app.on_event`.	main.py, requirements.txt
Medium	Supabase free tier: no backups ("No backups" on dashboard) and the project pauses after inactivity.	Supabase
Note	In `user_management`, `person_type='admin'` means **Staff** (shown as Staff in UI). The new Admin / Super Admin **login accounts must be a separate table** - do not reuse it.	models/person.py, routes.js
Good	RLS is enabled on all 9 tables (no anon access through Supabase REST). Upload limits exist (5 MB, 2000 rows, zip-size check).	Supabase, utils/spreadsheet.py

3. Phase 1 - Authentication & roles (foundation)

Goal: real login for two roles - **super_admin** and **admin** - and every API route protected. Phones running the scanner log in as an admin.

A-01	Accounts table (separate from user_management)
Build	- New table `admin_accounts`: id, email (unique, lower-cased), full_name, password_hash, role (super_admin | admin), status (active | disabled), must_change_password, failed_login_count, locked_until, last_login_at, created_by (FK self), created_at, updated_at. - SQLAlchemy model + creation in `app/schema.py` (same pattern as existing tables) + new migration `migrations/005_admin_accounts.sql` + update `full_schema.sql`. - Enable RLS on the new table (`ALTER TABLE ... ENABLE ROW LEVEL SECURITY`), no anon policies.
Done when	- Table exists on Supabase and on SQLite (tests); RLS enabled; migration file and full_schema.sql updated.

A-02	Password hashing & policy
Build	- Hash with Argon2id (argon2-cffi) or bcrypt; never store or log plain passwords. - Policy: min 10 characters, not equal to email, reject the 100 most common passwords.
Done when	- Unit tests prove hash/verify, wrong-password rejection, and policy rules.

A-03	Bootstrap first Super Admin (no default credentials)
Build	- CLI script `backend/scripts/create_super_admin.py`: prompts for email, name and password (hidden input), refuses if a super admin already exists unless `--force`. - Document it in README and SETUP_GUIDE.
Done when	- Running the script creates exactly one super admin; there is no hard-coded or seeded password anywhere in the repo.

A-04	Auth API
Build	- `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`, `POST /auth/change-password`, `POST /auth/refresh`. - Short-lived access token (15 min, JWT signed with `JWT_SECRET` from .env) + refresh token (7 days) in an **httpOnly, Secure, SameSite=Strict** cookie; refresh tokens stored hashed so logout/disable revokes them. - Lockout: 5 failed attempts -> locked 15 min. Rate limit login per IP (e.g. slowapi). Same generic message for unknown email and wrong password. - Disabled accounts cannot log in and their existing sessions stop working.
Done when	- Tests cover: success, wrong password, unknown email, lockout, disabled account, expired token, refresh, logout.

A-05	Protect every existing route
Build	- FastAPI dependencies `require_admin` (admin or super_admin) and `require_super_admin`. - Apply to ALL routers: users/people, courses, batches, timings, attendance, face, imports, sheet. Only `GET /health` (minimal) and `/auth/login` stay public. - Record `marked_by` on attendance from the logged-in account instead of client input.
Done when	- A test walks every registered route and asserts 401 without a token (except the public allow-list).

A-06	Frontend login & session
Build	- Real login form calling `/auth/login`; auth context with current user + role; route guards; automatic refresh; on 401 redirect to login; logout button in desktop and mobile layouts. - Forced "change password" screen when `must_change_password` is true. - Admins never see super-admin pages; super admins land on the Super Admin portal.
Done when	- Unauthenticated users cannot see any page except login; refreshing the browser keeps the session; logout clears it.

4. Phase 2 - Super Admin portal

SA-01	Portal shell
Build	- Route `/super-admin` with its own navigation: Dashboard, Admins, Issues, Audit Log, (link to normal console). - Same design system (Tailwind tokens, lucide icons, existing ui.jsx components); works on desktop and phone.
Done when	- Only super_admin can open it - direct URL as admin shows "not allowed" and the API returns 403.

SA-02	Admin management
Build	- List admins (search, filter by status/role), create admin (name, email, role), edit, disable / re-enable, reset password. - On create/reset: generate a strong temporary password shown **once**, set `must_change_password=true`. - Guards: cannot disable or demote yourself; cannot remove the last active super admin; email must be unique. - API under `/super-admin/admins`, all `require_super_admin`.
Done when	- New admin can log in with the temporary password, is forced to change it, then sees the normal console.

SA-03	Super Admin dashboard
Build	- Cards: active admins, open issues (by severity), issues resolved this week, today's attendance totals. - Recent activity list (from the audit log) and newest open issues with links.
Done when	- Numbers match the database; empty states shown when there is no data.

SA-04	Audit log
Build	- Table `audit_logs`: id, actor_id, action, target_type, target_id, details (JSON, no secrets/embeddings), ip, user_agent, created_at. RLS enabled. - Write entries for: login success/failure, logout, admin created/updated/disabled/password reset, issue created/status changed, person/course/batch deleted, attendance manually marked, face enrollment started/completed/cancelled, bulk upload. - Viewer with filters (actor, action, date range) and pagination. Read-only - no edit/delete endpoints.
Done when	- Each listed action produces one log row; admins cannot read the audit log.

5. Phase 3 - Issue reporting (Admin -> Super Admin)

IR-01	Data model
Build	- Table `issues`: id, title, description, category (bug | face_recognition | attendance_data | camera_device | login_access | other), severity (low | medium | high | critical), status (open | in_progress | resolved | closed), page_url, device_info (browser, OS, screen - captured automatically), reported_by (FK admin_accounts), assigned_to (nullable), resolution_note, created_at, updated_at, resolved_at. - Table `issue_comments`: id, issue_id, author_id, body, created_at. - Optional v1.1: one screenshot attachment (PNG/JPEG, max 2 MB, private Supabase Storage bucket, type checked by content not extension). - Migration `006_issues.sql`, full_schema.sql updated, RLS enabled on both tables.
Done when	- Tables exist with constraints (enum values checked by the API and a DB CHECK).

IR-02	Admin side - Report a problem
Build	- "Report a problem" button visible on every page (header on desktop, "More" menu on phone). Opens a form: title, category, severity, description; current page and device info attached automatically. - "My reports" page: own issues with status badge, comments thread, can add a comment; cannot change status. - Validation: title 5-120 chars, description 10-4000 chars; text is rendered as plain text (no HTML) everywhere.
Done when	- Admin submits an issue and sees it in My reports with status Open.

IR-03	Super Admin side - Issues inbox
Build	- Issues list with filters (status, severity, category, reporter, date) and sort (newest, severity). Open-issue count badge in the portal navigation, refreshed every 30-60 s. - Issue detail: full info, device info, comments, change status, assign, resolution note (required to resolve). - Status changes and comments are visible to the reporting admin.
Done when	- An issue reported by an admin appears in the Super Admin inbox within 60 s; resolving it updates the admin's My reports.

IR-04	Permissions
Build	- Admin: create issues, read/comment only their own. Super admin: read/update all. Enforced in the API (not only hidden in the UI).
Done when	- Admin requesting another admin's issue by ID gets 404/403 (IDOR test passes).

6. Phase 4 - Security hardening (production grade)

Ask Claude Code to run these as an audit first (report findings with file:line), then fix. Each row is one task.

ID	Area	Check / task
S-01	Secrets	Rotate Supabase DB password and service-role key; update backend/.env. Scan full git history for secrets (gitleaks). Confirm .env, *.db, model weights are gitignored. Add `JWT_SECRET` (32+ random bytes) to .env.example as a placeholder.
S-02	Least-privilege DB	Create a dedicated Postgres role for the app with only the needed table privileges (no superuser, no DDL at runtime in prod; run migrations with a separate role). Keep RLS on; no anon/authenticated policies.
S-03	AuthN / AuthZ	Every route protected (A-05 test). Role checks server-side. IDOR tests on users, faces, attendance, issues. Token expiry and revocation work. Logout invalidates refresh token.
S-04	Brute force / abuse	Rate limits: login, /face/recognize, /face/enrollment/frame, uploads. Account lockout. Max request body size (e.g. 2 MB for frames, 5 MB for uploads).
S-05	Error handling	Replace `str(exc)` in the global handler with a generic message + request ID; log details server-side only.
S-06	CORS & CSRF	Explicit allow-list of origins from env (no regex in prod). Cookies SameSite=Strict + Secure; add CSRF token if any cookie-authenticated state-changing request exists.
S-07	Security headers	CSP (self only, camera allowed), HSTS, X-Content-Type-Options nosniff, X-Frame-Options DENY / frame-ancestors none, Referrer-Policy, Permissions-Policy (camera=self, microphone=(), geolocation=()).
S-08	Input validation	Pydantic limits on all strings/numbers; SQL only through ORM or bound parameters (no f-strings in `text()`); output escaping in React (no dangerouslySetInnerHTML).
S-09	File uploads	Keep existing limits; validate by content; reject macros (.xlsm); protect CSV/Excel exports from formula injection (prefix =,+,-,@ with ').
S-10	Biometric data & privacy	No face images stored (verify). Embeddings deleted when a person is deleted (CASCADE). Consent field/notice at enrollment. Retention policy. Export/delete-my-data path. Align with India DPDP Act 2023. Embeddings never in logs or API responses.
S-11	Liveness / spoofing	Verify FACE_LIVENESS and challenge are on in production; test printed photo and phone-screen replay are rejected.
S-12	API surface	Disable /docs, /redoc, /openapi.json in production (env flag). /health returns only ok/not-ok publicly.
S-13	Dependencies	Pin versions (requirements.txt with exact versions or a lock file; package-lock committed). Run `pip-audit`, `npm audit`, `bandit -r app`; fix high/critical.
S-14	Transport	Real TLS certificate in production (reverse proxy); no self-signed; HTTP -> HTTPS redirect. DB connection with sslmode=require.
S-15	Logging	Structured logs with request ID, actor, action; no passwords, tokens, embeddings or full phone numbers. Log rotation.
S-16	Session UX	Idle timeout (e.g. 30 min on admin console), "logged out" message, re-login required after password change.
S-17	Pen-test pass	Run OWASP ZAP baseline scan against the staging URL; fix medium+ findings. Re-run the full manual security checklist (section 11).

7. Phase 5 - Compatibility

ID	Check / task
C-01	Desktop browsers (latest 2 versions): Chrome, Edge, Firefox, Safari (macOS). All pages, login, uploads, reports.
C-02	Phone scanner: Android Chrome, Samsung Internet, iOS Safari (iPhone - needs https, `playsinline`, user gesture to start camera). Front/back camera switch, portrait and landscape.
C-03	Screen widths: 360, 390, 768, 1024, 1280, 1440+ px - no horizontal scroll, tables scroll inside their container, touch targets >= 44 px.
C-04	Runtimes: Python 3.10.x pinned (document why), Node 20 and 22 LTS (and current 24), Windows 10 and 11; optional Linux server (Ubuntu 22.04/24.04) for production.
C-05	Databases: Supabase Postgres 17 (prod) and SQLite (tests) both pass the full test suite.
C-06	Framework deprecations: replace `@app.on_event("startup")` with FastAPI lifespan; check SQLAlchemy 2.x and Pydantic v2 warnings; React 19 console warnings clean.
C-07	Accessibility basics: keyboard navigation, visible focus, labels on all inputs, colour contrast AA, screen-reader names on icon buttons.
C-08	Low-end hardware: scanner usable on a 4 GB RAM laptop and a mid-range phone; frame size/quality settings documented.

8. Phase 6 - Availability & reliability

ID	Check / task
AV-01	Health: `/health/live` (process up) and `/health/ready` (DB reachable + face models ready). Frontend shows a clear banner when models are loading or DB is down (503 with message, not a crash).
AV-02	Database resilience: pool size/timeouts set; `pool_pre_ping` kept; retry on transient connection errors; works through the Supabase pooler if direct host fails.
AV-03	Supabase plan: free tier pauses after inactivity and has no backups - upgrade to Pro for production, or at minimum schedule a daily `pg_dump` to a safe location and document the restore steps. Test a restore.
AV-04	Production build: serve `npm run build` output from a reverse proxy (Caddy or Nginx) with real TLS, proxying /api to uvicorn. Do not use the Vite dev server in production.
AV-05	Process supervision: backend runs as a service that restarts on crash and on reboot (Windows: NSSM service; Linux: systemd; or Docker with restart policy). Optional Dockerfile + docker-compose.
AV-06	Model weights: pre-download DeepFace/YOLO weights during setup so startup does not depend on the internet; document the cache folder (~/.deepface).
AV-07	Load test: 3-5 phones scanning at once + admins using reports; measure /face/recognize latency and CPU/RAM; set FACE_TF_THREADS accordingly.
AV-08	Monitoring: uptime check on /health/ready, error alerts (e.g. log-based or Sentry), disk-space warning.
AV-09	Offline / network loss on scanner: clear message, no duplicate attendance when the connection returns (unique index already exists - verify).
AV-10	Runbook: one page - start/stop, where logs are, how to restore DB, how to rotate secrets, how to add a super admin.

9. Phase 7 - Tests & CI

ID	Check / task
T-01	Backend pytest: auth, roles matrix (anon / admin / super_admin x every route), admin management guards, issues permissions, audit log entries.
T-02	Frontend `node --test`: auth helpers, route guards, issue form validation.
T-03	GitHub Actions on every push/PR: backend tests (Python 3.10, SQLite), frontend tests + `npm run build`, pip-audit, npm audit, bandit, gitleaks.
T-04	Existing tests still pass (imports, phone validation, edit forms, recognition helpers).

10. Master prompt & phase prompts (copy into Claude Code)

Save this PDF in the repo as `docs/Vernex_Task_Sheet.pdf` so Claude Code can read it. Open the project folder in VS Code, start Claude Code, and paste the master prompt first. Then paste one phase prompt per session.

10.1 Master prompt (paste at the start of every session)

```
You are working on the Vernex Smart Attendance project (FastAPI + SQLAlchemy +
DeepFace backend in /backend, React 19 + Vite + Tailwind frontend in /frontend,
Supabase PostgreSQL via DATABASE_URL in backend/.env).

Read docs/Vernex_Task_Sheet.pdf fully, plus README.md, SETUP_GUIDE.md,
backend/app/main.py, backend/app/schema.py, backend/app/routes/,
frontend/src/routes.js, frontend/src/lib/api.js and frontend/src/layout.jsx
before changing anything.

Rules:
1. Work only on the phase I name. Follow the task IDs and their "Done when"
   criteria exactly. Do not start other phases.
2. Start in plan mode: list the files you will add/change per task, the new
   tables/endpoints/pages, and the tests. Wait for my approval.
3. Match the existing code style, naming, comment density and UI design
   (Tailwind tokens, ui.jsx components, lucide icons). Desktop and phone layouts.
4. Database changes: add a SQLAlchemy model, create it in app/schema.py, add a
   numbered SQL file in backend/migrations/, update full_schema.sql, and enable
   Row Level Security on every new table. Never drop or rewrite existing data.
   user_management.person_type='admin' means STAFF - login accounts live in
   the separate admin_accounts table.
5. Security: never hard-code or print secrets; read them from backend/.env and
   add placeholders to .env.example. Every new API route must require a role.
6. Write/extend tests for every task. Run: backend
   `.\.venv\Scripts\python -m pytest` and frontend `npm test` and
   `npm run build`. All must pass before you say a task is done.
7. Start the backend with $env:PYTHONUTF8="1" if you need to run it.
8. Do not commit or push unless I ask. When a phase is finished, give me:
   a summary per task ID, files changed, test results, and which manual
   checks from section 11 I should now run.
```

10.2 Phase prompts

```
PHASE 1: Implement Phase 1 - Authentication & roles (tasks A-01 to A-06) from
docs/Vernex_Task_Sheet.pdf. Plan first.
```

```
PHASE 2: Phase 1 is merged. Implement Phase 2 - Super Admin portal (SA-01 to
SA-04). Plan first. The portal must be unreachable for the admin role, both
in the UI and the API.
```

```
PHASE 3: Implement Phase 3 - Issue reporting (IR-01 to IR-04). Admins report
problems from any page; the issues must appear in the Super Admin portal
inbox with a live open-issue badge. Plan first.
```

```
PHASE 4: Run a full security audit using section 6 (S-01 to S-17) as the
checklist. First produce a findings report (severity, file:line, fix) without
changing code. After I approve, fix them in order of severity and add tests.
```

```
PHASE 5 + 6: Work through Compatibility (C-01 to C-08) and Availability
(AV-01 to AV-10). For items that need a real device, browser or server, write
the exact steps for me instead of guessing. Plan first.
```

```
PHASE 7: Implement Tests & CI (T-01 to T-04), including a GitHub Actions
workflow. Then run everything and report results.
```

```
FINAL REVIEW: Review the whole branch against every "Done when" criterion in
docs/Vernex_Task_Sheet.pdf and list anything incomplete. Then print the
section 11 manual-check list for me to sign off.
```

11. Manual checks (sign off before go-live)

Do these by hand, on real devices, after the matching phase. Tick each box and note any problem (report problems through the new Issue reporting feature once Phase 3 is live).

	Setup & run	Pass / Note
[ ]	Fresh clone on a second laptop: section 1.1 commands work without errors.	
[ ]	Connection test prints "Supabase PostgreSQL connection successful".	
[ ]	Backend /health shows database connected and face_models ready.	
[ ]	Frontend opens on https://localhost:5173 and on the phone via the Wi-Fi link.	
[ ]	start.bat works (or the PYTHONUTF8 note is fixed in start-backend.bat).	

	Login & roles	Pass / Note
[ ]	Wrong password -> generic error; correct password -> logged in.	
[ ]	5 wrong attempts -> account locked for 15 min; message is clear.	
[ ]	Opening any console URL while logged out redirects to login.	
[ ]	Calling an API URL without login (browser or curl) returns 401.	
[ ]	First login with a temporary password forces a password change.	
[ ]	Logout works on desktop and phone; Back button does not show data after logout.	
[ ]	Session survives a page refresh; expires after the idle timeout.	

	Super Admin portal	Pass / Note
[ ]	Super admin can create an admin; temporary password shown only once.	
[ ]	New admin can log in, must change password, then sees the normal console.	
[ ]	Disable an admin -> they are logged out and cannot log in; re-enable works.	
[ ]	Reset password works; old password stops working.	
[ ]	Cannot disable yourself or the last super admin.	
[ ]	Admin typing /super-admin in the address bar is blocked (page and API).	
[ ]	Dashboard numbers match the data; audit log shows each action above.	

	Issue reporting	Pass / Note
[ ]	Admin reports a problem from the desktop console (page URL + device info attached).	
[ ]	Admin reports a problem from the phone scanner page.	
[ ]	Issue appears in the Super Admin inbox within 60 s; badge count increases.	
[ ]	Super admin comments and changes status -> admin sees it in My reports.	
[ ]	Resolving requires a resolution note; resolved_at is set.	
[ ]	Admin cannot open another admin's issue by changing the ID in the URL.	
[ ]	Text with <script> or HTML is shown as plain text, not executed.	

	Existing features (regression)	Pass / Note
[ ]	Create course -> batch -> student; edit and delete work.	

	Existing features (regression)	Pass / Note
[ ]	Bulk upload (xlsx and csv) for courses, batches, students, staff, members; bad rows reported.	
[ ]	Face enrollment completes for a new student (quick and guided modes).	
[ ]	Live scanner marks a known student present / late; unknown face stays red.	
[ ]	Printed photo and phone-screen photo are rejected ("Photo detected").	
[ ]	Same student is not marked twice in one session/day.	
[ ]	Reports filter by course, batch and date; export opens correctly in Excel.	
[ ]	Existing demo data (DEMO-101) still visible after all migrations.	

	Security	Pass / Note
[ ]	Supabase DB password and service-role key rotated; old ones no longer work.	
[ ]	gitleaks / secret scan of the whole git history is clean.	
[ ]	App connects with the least-privilege DB role, not postgres.	
[ ]	Supabase REST with the anon key cannot read any table (RLS).	
[ ]	/docs and /openapi.json return 404 in production mode.	
[ ]	Server errors show a generic message + request ID (no SQL/stack trace).	
[ ]	Security headers present (check with browser DevTools or securityheaders.com).	
[ ]	Request from an unknown origin is refused by CORS.	
[ ]	pip-audit, npm audit, bandit: no high/critical findings open.	
[ ]	OWASP ZAP baseline scan: no medium+ findings open.	
[ ]	Deleting a student also deletes their face embeddings.	
[ ]	Logs contain no passwords, tokens or embeddings.	

	Compatibility	Pass / Note
[ ]	Chrome, Edge, Firefox, Safari (desktop): login, all pages, uploads, reports.	
[ ]	Android Chrome and Samsung Internet: scanner camera starts and recognises.	
[ ]	iPhone Safari: scanner camera starts (https, permission prompt) and recognises.	
[ ]	Phone portrait and landscape; widths 360 / 768 / 1280 px - no broken layout.	
[ ]	Keyboard-only navigation through login, forms and issue reporting.	

	Availability	Pass / Note
[ ]	Kill the backend process -> it restarts automatically.	
[ ]	Reboot the server PC -> backend and frontend come back without manual steps.	
[ ]	Disconnect internet briefly -> clear error, recovers when back, no duplicate attendance.	
[ ]	Backend restart while models load -> UI shows "loading" banner, not a crash.	
[ ]	Database backup taken and a test restore completed successfully.	
[ ]	3-5 phones scanning at the same time: recognition stays responsive.	
[ ]	Uptime monitor alerts when /health/ready fails.	

	Go-live	Pass / Note
[ ]	Production uses the built frontend behind a real TLS certificate (not the Vite dev server).	
[ ]	Supabase plan with backups and no auto-pause (or documented daily pg_dump).	
[ ]	Runbook written and stored with the team; at least two super admins exist.	
[ ]	All tests and CI green on the release commit; release tagged in git.	

Sign-off: _____ (Super Admin) Date: _____